

PDS2-2026-PF-grupo13

1.0

Generated by Doxygen 1.17.0

1 Sistema de Tolerância de Falha em Fábricas	1
1.1 Funcionalidades	1
1.2 Hierarquia do Sistema	1
1.3 Cargos de Usuário	2
1.4 Estrutura de Diretórios	2
1.5 Compilação	2
1.6 Documentação	2
1.7 Equipe	2
2 Directory Hierarchy	3
2.1 Directories	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Directory Documentation	9
5.1 include Directory Reference	9
6 Class Documentation	11
6.1 Alarme Class Reference	11
6.1.1 Detailed Description	11
6.1.2 Constructor & Destructor Documentation	12
6.1.2.1 Alarme()	12
6.1.3 Member Function Documentation	12
6.1.3.1 AtualizarAlarme()	12
6.1.3.2 GetEstado()	12
6.1.3.3 SetDesc()	13
6.1.3.4 SetEstado()	13
6.2 Conjunto Class Reference	13
6.2.1 Detailed Description	14
6.2.2 Constructor & Destructor Documentation	14
6.2.2.1 Conjunto()	14
6.2.3 Member Function Documentation	14
6.2.3.1 AdicionarEquipamento()	14
6.2.3.2 AtualizarAlarmes()	15
6.2.3.3 DiagnosticarConjunto()	15
6.2.3.4 ExibirAlarmes()	15
6.2.3.5 ExibirEquipamentos()	15
6.2.3.6 ExibirTudo()	16
6.2.3.7 GetDesc()	16
6.2.3.8 GetEquipamentos()	16

6.2.3.9 GetId()	16
6.2.3.10 GetNome()	16
6.2.3.11 QuantidadeAlarmes()	17
6.2.3.12 QuantidadeEquipamentos()	17
6.2.3.13 RemoverEquipamento()	17
6.2.3.14 SetDesc()	17
6.2.3.15 SetNome()	18
6.3 Equipamento Class Reference	18
6.3.1 Detailed Description	19
6.3.2 Constructor & Destructor Documentation	19
6.3.2.1 Equipamento()	19
6.3.3 Member Function Documentation	19
6.3.3.1 AdicionarParametro()	19
6.3.3.2 AdicionarParametroComLimites()	20
6.3.3.3 AtualizarAlarmes()	20
6.3.3.4 DiagnosticarEquipamento()	20
6.3.3.5 ExibirAlarmes()	21
6.3.3.6 ExibirParametros()	21
6.3.3.7 GetAlarmes()	21
6.3.3.8 GetId()	21
6.3.3.9 GetParametros()	21
6.3.3.10 QuantidadeAlarmes()	22
6.3.3.11 QuantidadeParametros()	22
6.3.3.12 RemoverParametro()	22
6.3.3.13 SetDesc()	22
6.4 LinhaDeProducao Class Reference	22
6.4.1 Detailed Description	23
6.4.2 Constructor & Destructor Documentation	23
6.4.2.1 LinhaDeProducao()	23
6.4.3 Member Function Documentation	24
6.4.3.1 AdicionarConjunto()	24
6.4.3.2 AtualizarAlarmes()	24
6.4.3.3 DiagnosticarLinha()	24
6.4.3.4 ExibirAlarmes()	25
6.4.3.5 ExibirConjuntos()	25
6.4.3.6 ExibirTudo()	25
6.4.3.7 GetConjuntos()	25
6.4.3.8 GetDesc()	25
6.4.3.9 GetId()	26
6.4.3.10 GetLocal()	26
6.4.3.11 GetNome()	26
6.4.3.12 QuantidadeAlarmes()	26

6.4.3.13 QuantidadeConjuntos()	26
6.4.3.14 RemoverConjunto()	27
6.4.3.15 SetDesc()	27
6.4.3.16 SetLocal()	27
6.4.3.17 SetNome()	27
6.5 Log Class Reference	28
6.5.1 Detailed Description	28
6.5.2 Constructor & Destructor Documentation	28
6.5.2.1 Log()	28
6.5.3 Member Function Documentation	29
6.5.3.1 GetArquivoLog()	29
6.5.3.2 Registrar()	29
6.5.3.3 SetArquivoLog()	29
6.6 Parametro Class Reference	30
6.6.1 Detailed Description	30
6.6.2 Constructor & Destructor Documentation	31
6.6.2.1 Parametro() [1/2]	31
6.6.2.2 Parametro() [2/2]	31
6.6.3 Member Function Documentation	31
6.6.3.1 DiagnosticarParametro()	31
6.6.3.2 GetDesc()	32
6.6.3.3 GetEstado()	32
6.6.3.4 GetHist()	32
6.6.3.5 GetId()	32
6.6.3.6 GetMax()	32
6.6.3.7 GetMin()	33
6.6.3.8 GetValorAtual()	33
6.6.3.9 ResetarHistorico()	33
6.6.3.10 ResetarLimites()	33
6.6.3.11 SetDesc()	33
6.6.3.12 SetMax()	34
6.6.3.13 SetMin()	34
6.6.3.14 SetValorAtual()	34
6.6.3.15 ValorValido()	35
6.7 Sistema Class Reference	35
6.7.1 Detailed Description	36
6.7.2 Constructor & Destructor Documentation	36
6.7.2.1 Sistema()	36
6.7.3 Member Function Documentation	36
6.7.3.1 AdicionarLinha()	36
6.7.3.2 AdicionarUsuario()	37
6.7.3.3 CarregarSave()	37

6.7.3.4 CarregarUltimoSave()	37
6.7.3.5 ExibirLinhas()	38
6.7.3.6 GetLinhas()	38
6.7.3.7 GetUsuarioLogado()	38
6.7.3.8 IdDisponivel()	38
6.7.3.9 Login()	39
6.7.3.10 Logout()	39
6.7.3.11 ProximoldDisponivel()	39
6.7.3.12 RemoverLinha()	40
6.7.3.13 RemoverUsuario()	40
6.7.3.14 SalvarAlteracoes()	40
6.8 Usuario Class Reference	41
6.8.1 Detailed Description	41
6.8.2 Constructor & Destructor Documentation	41
6.8.2.1 Usuario()	41
6.8.3 Member Function Documentation	42
6.8.3.1 GetCargo()	42
6.8.3.2 GetLogin()	42
6.8.3.3 ValidarSenha()	42
7 File Documentation	43
7.1 include/Alarme.hpp File Reference	43
7.2 Alarme.hpp	43
7.3 include/Cargo.hpp File Reference	44
7.3.1 Enumeration Type Documentation	44
7.3.1.1 Cargo	44
7.4 Cargo.hpp	44
7.5 include/Conjunto.hpp File Reference	44
7.6 Conjunto.hpp	45
7.7 include/Equipamento.hpp File Reference	45
7.8 Equipamento.hpp	46
7.9 include/LinhaDeProducao.hpp File Reference	46
7.10 LinhaDeProducao.hpp	46
7.11 include/Log.hpp File Reference	47
7.12 Log.hpp	47
7.13 include/Parametro.hpp File Reference	48
7.14 Parametro.hpp	48
7.15 include/Sistema.hpp File Reference	49
7.16 Sistema.hpp	49
7.17 include/Usuario.hpp File Reference	50
7.18 Usuario.hpp	50
7.19 README.md File Reference	50

Chapter 1

Sistema de Tolerância de Falha em Fábricas

[Sistema](#) de monitoramento industrial orientado a objetos desenvolvido em C++. Permite o acompanhamento em tempo real de parâmetros de equipamentos organizados em linhas de produção, detectando automaticamente condições de falha e disparando alarmes.

1.1 Funcionalidades

- Gerenciamento hierárquico de **Linhas de Produção** → **Conjuntos** → **Equipamentos** → **Parâmetros**
- Monitoramento de parâmetros com limites configuráveis (Min/Max)
- Disparo automático de **alarmes** quando parâmetros violam seus limites
- **Diagnóstico** propagado por toda a hierarquia, identificando os pontos de falha
- Controle de acesso por usuários com três níveis de cargo
- Persistência de dados com save/load de estado
- Registro de eventos via [Log](#) em arquivo

1.2 Hierarquia do Sistema

```
Sistema
|-- LinhaDeProducao
    |-- Conjunto
        |-- Equipamento
            +-- Parametro
            |-- Alarme
```

1.3 Cargos de Usuário

Cargo	Permissões
ADMIN	Gerencia usuários e toda a estrutura do sistema
ENGENHEIRO	Configura linhas, conjuntos e equipamentos
TECNICO	Visualiza dados e alarmes

1.4 Estrutura de Diretórios

```
.
+-- include/      # Contratos das classes (.hpp)
+-- src/          # Implementacoes (.cpp)
+-- design/       # User Stories e Cartoes CRC
+-- tests/        # Testes automatizados
+-- build/        # Artefatos de compilacao
+-- docs/         # Documentacao gerada pelo Doxygen
\-- README.md
```

1.5 Compilação

```
make
```

1.6 Documentação

A documentação das classes pode ser gerada com o Doxygen a partir da raiz do projeto:

```
doxygen Doxyfile
```

A saída HTML estará disponível em `docs/html/index.html`. Uma versão, atualiza, em PDF pode ser encontrada na raiz do projeto como `doxygen_doc.pdf`.

1.7 Equipe

Nº Matrícula	Nome
2024010002	Felipe Hildegardes Jorge
2022028400dox	Wennedes de Oliveira Nogueira Junior
2020057144	Gustavo Coelho Santos

Chapter 2

Directory Hierarchy

2.1 Directories

include	9
Alarme.hpp	43
Cargo.hpp	44
Conjunto.hpp	44
Equipamento.hpp	45
LinhaDeProducao.hpp	46
Log.hpp	47
Parametro.hpp	48
Sistema.hpp	49
Usuario.hpp	50

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Alarme		
	Alarme associado a um Parametro de um equipamento	11
Conjunto		
	Agrupamento lógico de equipamentos dentro de uma LinhaDeProducao	13
Equipamento		
	Representa um equipamento físico dentro de um Conjunto	18
LinhaDeProducao		
	Representa uma linha de produção industrial	22
Log		
	Registra eventos e operações do sistema em arquivo de texto	28
Parametro		
	Representa um parâmetro monitorado de um equipamento	30
Sistema		
	Ponto de entrada central do sistema de monitoramento	35
Usuario		
	Representa um usuário do sistema com credenciais e cargo	41

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

include/ Alarme.hpp	43
include/ Cargo.hpp	44
include/ Conjunto.hpp	44
include/ Equipamento.hpp	45
include/ LinhaDeProducao.hpp	46
include/ Log.hpp	47
include/ Parametro.hpp	48
include/ Sistema.hpp	49
include/ Usuario.hpp	50

Chapter 5

Directory Documentation

5.1 include Directory Reference

Files

- file [Alarme.hpp](#)
- file [Cargo.hpp](#)
- file [Conjunto.hpp](#)
- file [Equipamento.hpp](#)
- file [LinhaDeProducao.hpp](#)
- file [Log.hpp](#)
- file [Parametro.hpp](#)
- file [Sistema.hpp](#)
- file [Usuario.hpp](#)

Chapter 6

Class Documentation

6.1 Alarme Class Reference

[Alarme](#) associado a um [Parametro](#) de um equipamento.

```
#include <Alarme.hpp>
```

Public Member Functions

- [Alarme](#) (std::string Desc, int Id, [Parametro](#) &p)
Constrói um alarme vinculado a um parâmetro.
- bool [AtualizarAlarme](#) ()
Consulta o parâmetro associado e atualiza o estado do alarme.
- bool [GetEstado](#) () const
- void [SetEstado](#) (bool Estado)
Força o estado do alarme manualmente.
- void [SetDesc](#) (std::string desc)
Atualiza a descrição do alarme.

6.1.1 Detailed Description

[Alarme](#) associado a um [Parametro](#) de um equipamento.

Monitora um único parâmetro e mantém um estado booleano que indica se o valor atual viola os limites configurados naquele parâmetro.

Definition at line 14 of file [Alarme.hpp](#).

6.1.2 Constructor & Destructor Documentation

6.1.2.1 Alarme()

```
Alarme::Alarme (
    std::string Desc,
    int Id,
    Parametro & p)
```

Constrói um alarme vinculado a um parâmetro.

Parameters

<i>Desc</i>	Descrição do alarme.
<i>Id</i>	Identificador único do alarme.
<i>p</i>	Parâmetro monitorado por este alarme.

Note

Implementa US-014 (Criar [Alarme](#)).

6.1.3 Member Function Documentation

6.1.3.1 AtualizarAlarme()

```
bool Alarme::AtualizarAlarme ()
```

Consulta o parâmetro associado e atualiza o estado do alarme.

Returns

true se o alarme estiver ativo (parâmetro em falha).

Note

Implementa US-014 (Criar [Alarme](#)).

6.1.3.2 GetEstado()

```
bool Alarme::GetEstado () const
```

Returns

true se o alarme estiver ativo.

6.1.3.3 SetDesc()

```
void Alarme::SetDesc (
    std::string desc)
```

Atualiza a descrição do alarme.

Parameters

<i>desc</i>	Nova descrição.
-------------	-----------------

6.1.3.4 SetEstado()

```
void Alarme::SetEstado (
    bool Estado)
```

Força o estado do alarme manualmente.

Parameters

<i>Estado</i>	Novo estado.
---------------	--------------

The documentation for this class was generated from the following file:

- include/[Alarme.hpp](#)

6.2 Conjunto Class Reference

Agrupamento lógico de equipamentos dentro de uma [LinhaDeProducao](#).

```
#include <Conjunto.hpp>
```

Public Member Functions

- [Conjunto](#) (int id, std::string desc)
Constrói um conjunto.
- void [AdicionarEquipamento](#) (int id, std::string desc)
Adiciona um equipamento ao conjunto.
- void [RemoverEquipamento](#) (int id)
Remove o equipamento com o ID fornecido.
- void [SetDesc](#) (std::string desc)
Atualiza a descrição do conjunto.
- void [SetNome](#) (std::string nome)
Atualiza o nome do conjunto.
- void [ExibirEquipamentos](#) () const
Exibe no console os equipamentos do conjunto.
- void [ExibirAlarmes](#) () const
Exibe no console os alarmes ativos de todos os equipamentos.

- void [ExibirTudo](#) () const
Exibe equipamentos e alarmes de forma consolidada.
- int [QuantidadeEquipamentos](#) () const
- int [QuantidadeAlarmes](#) () const
- std::vector< int > [DiagnosticarConjunto](#) () const
Diagnostica todos os equipamentos do conjunto.
- void [AtualizarAlarmes](#) ()
Propaga a atualização de alarmes para todos os equipamentos.
- const std::vector< [Equipamento](#) > & [GetEquipamentos](#) () const
- int [GetId](#) () const
- std::string [GetDesc](#) () const
- std::string [GetNome](#) () const

6.2.1 Detailed Description

Agrupamento lógico de equipamentos dentro de uma [LinhaDeProducao](#).

Um conjunto reúne equipamentos correlacionados e propaga operações de diagnóstico e atualização de alarmes para todos eles.

Definition at line 14 of file [Conjunto.hpp](#).

6.2.2 Constructor & Destructor Documentation

6.2.2.1 Conjunto()

```
Conjunto::Conjunto (
    int id,
    std::string desc)
```

Constrói um conjunto.

Parameters

<i>id</i>	Identificador único.
<i>desc</i>	Descrição do conjunto.

6.2.3 Member Function Documentation

6.2.3.1 AdicionarEquipamento()

```
void Conjunto::AdicionarEquipamento (
    int id,
    std::string desc)
```

Adiciona um equipamento ao conjunto.

Parameters

<i>id</i>	Identificador único do equipamento.
<i>desc</i>	Descrição do equipamento.

Note

Implementa US-007 (Criar Equipamentos).

6.2.3.2 AtualizarAlarmes()

```
void Conjunto::AtualizarAlarmes ()
```

Propaga a atualização de alarmes para todos os equipamentos.

Note

Implementa US-014 (Criar [Alarme](#)).

6.2.3.3 DiagnosticarConjunto()

```
std::vector< int > Conjunto::DiagnosticarConjunto () const
```

Diagnostica todos os equipamentos do conjunto.

Returns

Vetor com IDs dos parâmetros em estado de falha.

Note

Implementa US-012 (Diagnóstico de [Conjunto](#)).

6.2.3.4 ExibirAlarmes()

```
void Conjunto::ExibirAlarmes () const
```

Exibe no console os alarmes ativos de todos os equipamentos.

6.2.3.5 ExibirEquipamentos()

```
void Conjunto::ExibirEquipamentos () const
```

Exibe no console os equipamentos do conjunto.

Note

Implementa US-006 (Acessar [Conjunto](#)).

6.2.3.6 ExibirTudo()

```
void Conjunto::ExibirTudo () const
```

Exibe equipamentos e alarmes de forma consolidada.

Note

Implementa US-006 (Acessar [Conjunto](#)).

6.2.3.7 GetDesc()

```
std::string Conjunto::GetDesc () const
```

Returns

Descrição do conjunto.

6.2.3.8 GetEquipamentos()

```
const std::vector< Equipamento > & Conjunto::GetEquipamentos () const
```

Returns

Referência constante ao vetor de equipamentos.

6.2.3.9 GetId()

```
int Conjunto::GetId () const
```

Returns

Identificador único do conjunto.

6.2.3.10 GetNome()

```
std::string Conjunto::GetNome () const
```

Returns

Nome do conjunto.

6.2.3.11 QuantidadeAlarmes()

```
int Conjunto::QuantidadeAlarmes () const
```

Returns

Número total de alarmes ativos no conjunto.

6.2.3.12 QuantidadeEquipamentos()

```
int Conjunto::QuantidadeEquipamentos () const
```

Returns

Número de equipamentos cadastrados no conjunto.

6.2.3.13 RemoverEquipamento()

```
void Conjunto::RemoverEquipamento (
    int id)
```

Remove o equipamento com o ID fornecido.

Parameters

<i>id</i>	Identificador do equipamento a remover.
-----------	---

Note

Implementa US-008 (Remover [Equipamento](#)).

6.2.3.14 SetDesc()

```
void Conjunto::SetDesc (
    std::string desc)
```

Atualiza a descrição do conjunto.

Parameters

<i>desc</i>	Nova descrição.
-------------	-----------------

6.2.3.15 SetNome()

```
void Conjunto::SetNome (
    std::string nome)
```

Atualiza o nome do conjunto.

Parameters

<i>nome</i>	Novo nome.
-------------	------------

The documentation for this class was generated from the following file:

- [include/Conjunto.hpp](#)

6.3 Equipamento Class Reference

Representa um equipamento físico dentro de um [Conjunto](#).

```
#include <Equipamento.hpp>
```

Public Member Functions

- [Equipamento](#) (int id, std::string desc)
Constrói um equipamento.
- void [AdicionarParametro](#) (double ValorAtual, std::string Desc, int id)
Adiciona um parâmetro sem limites definidos.
- void [AdicionarParametroComLimites](#) (double ValorAtual, double Max, double Min, std::string Desc)
Adiciona um parâmetro com limites Min/Max definidos.
- void [RemoverParametro](#) (int id)
Remove o parâmetro com o ID fornecido.
- std::vector< int > [DiagnosticarEquipamento](#) () const
Diagnostica todos os parâmetros do equipamento.
- void [AtualizarAlarmes](#) ()
Atualiza o estado de todos os alarmes do equipamento.
- void [ExibirParametros](#) () const
Exibe no console os parâmetros e seus valores atuais.
- void [ExibirAlarmes](#) () const
Exibe no console os alarmes ativos do equipamento.
- int [QuantidadeParametros](#) () const
- int [QuantidadeAlarmes](#) () const
- int [GetId](#) () const
- const std::vector< [Parametro](#) > & [GetParametros](#) () const
- const std::vector< [Alarme](#) > & [GetAlarmes](#) () const
- void [SetDesc](#) (std::string desc)
Atualiza a descrição do equipamento.

6.3.1 Detailed Description

Representa um equipamento físico dentro de um [Conjunto](#).

Gerencia uma coleção de parâmetros monitorados e os alarmes derivados deles. Oferece diagnóstico consolidado do estado do equipamento.

Definition at line 15 of file [Equipamento.hpp](#).

6.3.2 Constructor & Destructor Documentation

6.3.2.1 Equipamento()

```
Equipamento::Equipamento (
    int id,
    std::string desc)
```

Constrói um equipamento.

Parameters

<i>id</i>	Identificador único.
<i>desc</i>	Descrição do equipamento.

6.3.3 Member Function Documentation

6.3.3.1 AdicionarParametro()

```
void Equipamento::AdicionarParametro (
    double ValorAtual,
    std::string Desc,
    int id)
```

Adiciona um parâmetro sem limites definidos.

Parameters

<i>ValorAtual</i>	Leitura inicial.
<i>Desc</i>	Descrição do parâmetro.
<i>id</i>	Identificador único do parâmetro.

Note

Implementa US-018 (Configurar parâmetros de equipamentos).

6.3.3.2 AdicionarParametroComLimites()

```
void Equipamento::AdicionarParametroComLimites (
    double ValorAtual,
    double Max,
    double Min,
    std::string Desc)
```

Adiciona um parâmetro com limites Min/Max definidos.

Parameters

<i>ValorAtual</i>	Leitura inicial.
<i>Max</i>	Limite superior aceitável.
<i>Min</i>	Limite inferior aceitável.
<i>Desc</i>	Descrição do parâmetro.

Note

Implementa US-018 (Configurar parâmetros de equipamentos).

6.3.3.3 AtualizarAlarmes()

```
void Equipamento::AtualizarAlarmes ()
```

Atualiza o estado de todos os alarmes do equipamento.

Note

Implementa US-014 (Criar [Alarme](#)).

6.3.3.4 DiagnosticarEquipamento()

```
std::vector< int > Equipamento::DiagnosticarEquipamento () const
```

Diagnostica todos os parâmetros do equipamento.

Returns

Vetor com IDs dos parâmetros em estado de falha.

Note

Implementa US-011 (Diagnóstico de [Equipamento](#)).

6.3.3.5 ExibirAlarmes()

```
void Equipamento::ExibirAlarmes () const
```

Exibe no console os alarmes ativos do equipamento.

Note

Implementa US-009 (Acessar [Equipamento](#)).

6.3.3.6 ExibirParametros()

```
void Equipamento::ExibirParametros () const
```

Exibe no console os parâmetros e seus valores atuais.

Note

Implementa US-009 (Acessar [Equipamento](#)).

6.3.3.7 GetAlarmes()

```
const std::vector< Alarme > & Equipamento::GetAlarmes () const
```

Returns

Referência constante ao vetor de alarmes.

6.3.3.8 GetId()

```
int Equipamento::GetId () const
```

Returns

Identificador único do equipamento.

6.3.3.9 GetParametros()

```
const std::vector< Parametro > & Equipamento::GetParametros () const
```

Returns

Referência constante ao vetor de parâmetros.

6.3.3.10 QuantidadeAlarmes()

```
int Equipamento::QuantidadeAlarmes () const
```

Returns

Número de alarmes ativos.

6.3.3.11 QuantidadeParametros()

```
int Equipamento::QuantidadeParametros () const
```

Returns

Número de parâmetros cadastrados.

6.3.3.12 RemoverParametro()

```
void Equipamento::RemoverParametro (
    int id)
```

Remove o parâmetro com o ID fornecido.

Parameters

<i>id</i>	Identificador do parâmetro a remover.
-----------	---------------------------------------

6.3.3.13 SetDesc()

```
void Equipamento::SetDesc (
    std::string desc)
```

Atualiza a descrição do equipamento.

Parameters

<i>desc</i>	Nova descrição.
-------------	-----------------

The documentation for this class was generated from the following file:

- [include/Equipamento.hpp](#)

6.4 LinhaDeProducao Class Reference

Representa uma linha de produção industrial.

```
#include <LinhaDeProducao.hpp>
```

Public Member Functions

- [LinhaDeProducao](#) (int id, std::string desc, std::string local, std::string nome)
Constrói uma linha de produção.
- void [AdicionarConjunto](#) (int id, std::string desc)
Adiciona um conjunto à linha.
- void [RemoverConjunto](#) (int id)
Remove o conjunto com o ID fornecido.
- void [ExibirConjuntos](#) () const
Exibe no console os conjuntos da linha.
- void [ExibirAlarmes](#) () const
Exibe no console os alarmes ativos de toda a linha.
- void [ExibirTudo](#) () const
Exibe conjuntos, equipamentos e alarmes de forma consolidada.
- int [QuantidadeConjuntos](#) () const
- int [QuantidadeAlarmes](#) () const
- std::vector< int > [DiagnosticarLinha](#) () const
Diagnostica todos os conjuntos da linha.
- void [AtualizarAlarmes](#) ()
Propaga a atualização de alarmes para todos os conjuntos.
- void [SetDesc](#) (std::string desc)
Atualiza a descrição da linha.
- void [SetLocal](#) (std::string local)
Atualiza a localização da linha.
- void [SetName](#) (std::string nome)
Atualiza o nome da linha.
- const std::vector< [Conjunto](#) > & [GetConjuntos](#) () const
- int [GetId](#) () const
- std::string [GetDesc](#) () const
- std::string [GetLocal](#) () const
- std::string [GetName](#) () const

6.4.1 Detailed Description

Representa uma linha de produção industrial.

Agrega conjuntos de equipamentos e oferece operações de diagnóstico e visualização para toda a linha.

Definition at line 15 of file [LinhaDeProducao.hpp](#).

6.4.2 Constructor & Destructor Documentation

6.4.2.1 LinhaDeProducao()

```
LinhaDeProducao::LinhaDeProducao (  
    int id,  
    std::string desc,  
    std::string local,  
    std::string nome)
```

Constrói uma linha de produção.

Parameters

<i>id</i>	Identificador único.
<i>desc</i>	Descrição da linha.
<i>local</i>	Localização física.
<i>nome</i>	Nome da linha.

6.4.3 Member Function Documentation

6.4.3.1 AdicionarConjunto()

```
void LinhaDeProducao::AdicionarConjunto (
    int id,
    std::string desc)
```

Adiciona um conjunto à linha.

Parameters

<i>id</i>	Identificador único do conjunto.
<i>desc</i>	Descrição do conjunto.

Note

Implementa US-004 (Criar Conjuntos).

6.4.3.2 AtualizarAlarmes()

```
void LinhaDeProducao::AtualizarAlarmes ()
```

Propaga a atualização de alarmes para todos os conjuntos.

Note

Implementa US-014 (Criar [Alarme](#)).

6.4.3.3 DiagnosticarLinha()

```
std::vector< int > LinhaDeProducao::DiagnosticarLinha () const
```

Diagnostica todos os conjuntos da linha.

Returns

Vetor com IDs dos parâmetros em estado de falha.

Note

Implementa US-013 (Diagnóstico de Linha de Produção).

6.4.3.4 ExibirAlarmes()

```
void LinhaDeProducao::ExibirAlarmes () const
```

Exibe no console os alarmes ativos de toda a linha.

6.4.3.5 ExibirConjuntos()

```
void LinhaDeProducao::ExibirConjuntos () const
```

Exibe no console os conjuntos da linha.

Note

Implementa US-003 (Acessar Linhas de Produção).

6.4.3.6 ExibirTudo()

```
void LinhaDeProducao::ExibirTudo () const
```

Exibe conjuntos, equipamentos e alarmes de forma consolidada.

Note

Implementa US-003 (Acessar Linhas de Produção).

6.4.3.7 GetConjuntos()

```
const std::vector< Conjunto > & LinhaDeProducao::GetConjuntos () const
```

Returns

Referência constante ao vetor de conjuntos.

6.4.3.8 GetDesc()

```
std::string LinhaDeProducao::GetDesc () const
```

Returns

Descrição da linha.

6.4.3.9 GetId()

```
int LinhaDeProducao::GetId () const
```

Returns

Identificador único da linha.

6.4.3.10 GetLocal()

```
std::string LinhaDeProducao::GetLocal () const
```

Returns

Localização física da linha.

6.4.3.11 GetNome()

```
std::string LinhaDeProducao::GetNome () const
```

Returns

Nome da linha.

6.4.3.12 QuantidadeAlarmes()

```
int LinhaDeProducao::QuantidadeAlarmes () const
```

Returns

Número total de alarmes ativos na linha.

6.4.3.13 QuantidadeConjuntos()

```
int LinhaDeProducao::QuantidadeConjuntos () const
```

Returns

Número de conjuntos cadastrados na linha.

6.4.3.14 RemoveConjunto()

```
void LinhaDeProducao::RemoveConjunto (
    int id)
```

Remove o conjunto com o ID fornecido.

Parameters

<i>id</i>	Identificador do conjunto a remover.
-----------	--------------------------------------

Note

Implementa US-005 (Remover [Conjunto](#)).

6.4.3.15 SetDesc()

```
void LinhaDeProducao::SetDesc (
    std::string desc)
```

Atualiza a descrição da linha.

Parameters

<i>desc</i>	Nova descrição.
-------------	-----------------

6.4.3.16 SetLocal()

```
void LinhaDeProducao::SetLocal (
    std::string local)
```

Atualiza a localização da linha.

Parameters

<i>local</i>	Nova localização.
--------------	-------------------

6.4.3.17 SetNome()

```
void LinhaDeProducao::SetNome (
    std::string nome)
```

Atualiza o nome da linha.

Parameters

<i>nome</i>	Novo nome.
-------------	------------

The documentation for this class was generated from the following file:

- include/[LinhaDeProducao.hpp](#)

6.5 Log Class Reference

Registra eventos e operações do sistema em arquivo de texto.

```
#include <Log.hpp>
```

Public Member Functions

- [Log](#) (const std::string &arquivo)
Constrói o logger apontando para o arquivo especificado.
- bool [Registrar](#) (const std::string &mensagem) const
Acrescenta uma mensagem ao arquivo de log.
- std::string [GetArquivoLog](#) () const
- bool [SetArquivoLog](#) (const std::string &arquivo)
Redireciona o log para um novo arquivo.

6.5.1 Detailed Description

Registra eventos e operações do sistema em arquivo de texto.

Definition at line 9 of file [Log.hpp](#).

6.5.2 Constructor & Destructor Documentation

6.5.2.1 Log()

```
Log::Log (  
    const std::string & arquivo)
```

Constrói o logger apontando para o arquivo especificado.

Parameters

<i>arquivo</i>	Caminho do arquivo de log.
----------------	----------------------------

6.5.3 Member Function Documentation

6.5.3.1 GetArquivoLog()

```
std::string Log::GetArquivoLog () const
```

Returns

Caminho do arquivo de log atual.

6.5.3.2 Registrar()

```
bool Log::Registrar (  
    const std::string & mensagem) const
```

Acrescenta uma mensagem ao arquivo de log.

Parameters

<i>mensagem</i>	Texto a registrar (limite de 500 caracteres).
-----------------	---

Returns

true se o registro foi bem-sucedido.

Note

Implementa US-019 (Registro de informações no log).

6.5.3.3 SetArquivoLog()

```
bool Log::SetArquivoLog (  
    const std::string & arquivo)
```

Redireciona o log para um novo arquivo.

Parameters

<i>arquivo</i>	Novo caminho de arquivo.
----------------	--------------------------

Returns

true se a alteração foi aceita.

The documentation for this class was generated from the following file:

- include/[Log.hpp](#)

6.6 Parametro Class Reference

Representa um parâmetro monitorado de um equipamento.

```
#include <Parametro.hpp>
```

Public Member Functions

- [Parametro](#) (double ValorAtual, std::string Desc, int id)
Constrói um parâmetro sem limites definidos.
- [Parametro](#) (double ValorAtual, double Max, double Min, std::string Desc, int id)
Constrói um parâmetro com limites Min/Max definidos.
- bool [ValorValido](#) (double Valor) const
Verifica se um valor está dentro dos limites configurados.
- void [SetValorAtual](#) (double Valor)
Atualiza o valor atual e registra no histórico.
- void [SetMax](#) (double Valor)
Define o limite superior.
- void [SetMin](#) (double Valor)
Define o limite inferior.
- void [SetDesc](#) (std::string Desc)
Atualiza a descrição do parâmetro.
- bool [DiagnosticarParametro](#) ()
Verifica se o valor atual viola os limites e atualiza o estado.
- double [GetValorAtual](#) () const
- double [GetMax](#) () const
- double [GetMin](#) () const
- std::string [GetDesc](#) () const
- bool [GetEstado](#) () const
- const std::vector< double > & [GetHist](#) () const
- int [GetId](#) ()
- void [ResetarLimites](#) ()
Restaura os limites Min/Max para os valores padrão (sem restrição).
- void [ResetarHistorico](#) ()
Limpa o histórico de leituras.

6.6.1 Detailed Description

Representa um parâmetro monitorado de um equipamento.

Armazena um valor numérico em tempo real, limites aceitáveis (Min/Max), histórico de leituras e um estado de diagnóstico.

Definition at line 13 of file [Parametro.hpp](#).

6.6.2 Constructor & Destructor Documentation

6.6.2.1 Parametro() [1/2]

```
Parametro::Parametro (
    double ValorAtual,
    std::string Desc,
    int id)
```

Constrói um parâmetro sem limites definidos.

Parameters

<i>ValorAtual</i>	Leitura inicial do parâmetro.
<i>Desc</i>	Descrição do parâmetro.
<i>id</i>	Identificador único.

6.6.2.2 Parametro() [2/2]

```
Parametro::Parametro (
    double ValorAtual,
    double Max,
    double Min,
    std::string Desc,
    int id)
```

Constrói um parâmetro com limites Min/Max definidos.

Parameters

<i>ValorAtual</i>	Leitura inicial do parâmetro.
<i>Max</i>	Limite superior aceitável.
<i>Min</i>	Limite inferior aceitável.
<i>Desc</i>	Descrição do parâmetro.
<i>id</i>	Identificador único.

Note

Implementa US-018 (Configurar parâmetros de equipamentos).

6.6.3 Member Function Documentation

6.6.3.1 DiagnosticarParametro()

```
bool Parametro::DiagnosticarParametro ()
```

Verifica se o valor atual viola os limites e atualiza o estado.

Returns

true se o parâmetro estiver fora dos limites (em falha).

Note

Implementa US-011 (Diagnóstico de [Equipamento](#)).

6.6.3.2 GetDesc()

```
std::string Parametro::GetDesc () const
```

Returns

Descrição do parâmetro.

6.6.3.3 GetEstado()

```
bool Parametro::GetEstado () const
```

Returns

true se o parâmetro estiver em estado de falha.

6.6.3.4 GetHist()

```
const std::vector< double > & Parametro::GetHist () const
```

Returns

Histórico de leituras anteriores.

Note

Relacionado a US-010 (Leitura do [Equipamento](#)).

6.6.3.5 GetId()

```
int Parametro::GetId ()
```

Returns

Identificador único do parâmetro.

6.6.3.6 GetMax()

```
double Parametro::GetMax () const
```

Returns

Limite máximo configurado.

6.6.3.7 GetMin()

```
double Parametro::GetMin () const
```

Returns

Limite mínimo configurado.

6.6.3.8 GetValorAtual()

```
double Parametro::GetValorAtual () const
```

Returns

Valor atual do parâmetro.

6.6.3.9 ResetarHistorico()

```
void Parametro::ResetarHistorico ()
```

Limpa o histórico de leituras.

6.6.3.10 ResetarLimites()

```
void Parametro::ResetarLimites ()
```

Restaura os limites Min/Max para os valores padrão (sem restrição).

6.6.3.11 SetDesc()

```
void Parametro::SetDesc (  
    std::string Desc)
```

Atualiza a descrição do parâmetro.

Parameters

<i>Desc</i>	Nova descrição.
-------------	-----------------

6.6.3.12 SetMax()

```
void Parametro::SetMax (  
    double Valor)
```

Define o limite superior.

Parameters

<i>Valor</i>	Novo limite máximo.
--------------	---------------------

Note

Implementa US-018 (Configurar parâmetros de equipamentos).

6.6.3.13 SetMin()

```
void Parametro::SetMin (  
    double Valor)
```

Define o limite inferior.

Parameters

<i>Valor</i>	Novo limite mínimo.
--------------	---------------------

Note

Implementa US-018 (Configurar parâmetros de equipamentos).

6.6.3.14 SetValorAtual()

```
void Parametro::SetValorAtual (  
    double Valor)
```

Atualiza o valor atual e registra no histórico.

Parameters

<i>Valor</i>	Novo valor lido.
--------------	------------------

Note

Implementa US-010 (Leitura do [Equipamento](#)).

6.6.3.15 ValorValido()

```
bool Parametro::ValorValido (
    double Valor) const
```

Verifica se um valor está dentro dos limites configurados.

Parameters

<i>Valor</i>	Valor a verificar.
--------------	--------------------

Returns

true se o valor estiver dentro dos limites.

The documentation for this class was generated from the following file:

- include/[Parametro.hpp](#)

6.7 Sistema Class Reference

Ponto de entrada central do sistema de monitoramento.

```
#include <Sistema.hpp>
```

Public Member Functions

- [Sistema](#) ()
Inicializa o sistema sem linhas, usuários ou sessão ativa.
- void [AdicionarLinha](#) (int id, std::string desc, std::string local, std::string nome)
Cadastra uma nova linha de produção.
- void [RemoverLinha](#) (int id)
Remove a linha de produção com o ID fornecido.
- const std::vector< [LinhaDeProducao](#) > & [GetLinhas](#) () const
- void [ExibirLinhas](#) ()
Exibe no console todas as linhas de produção cadastradas.
- void [AdicionarUsuario](#) (std::string login, std::string senha, [Cargo](#) cargo)
Cadastra um novo usuário no sistema.
- void [RemoverUsuario](#) (std::string login)
Remove o usuário identificado pelo login.
- int [ProximoldDisponivel](#) () const
Retorna o próximo ID disponível para cadastro.
- bool [IdDisponivel](#) (int Id) const
Verifica se um ID está disponível para uso.
- bool [Login](#) (std::string login, std::string senha)
Autentica um usuário e inicia a sessão.
- bool [Logout](#) ()
Encerra a sessão do usuário atual.
- [Usuario](#) * [GetUsuarioLogado](#) () const
- bool [SalvarAlteracoes](#) ()
Persiste o estado atual do sistema em arquivo.
- bool [CarregarUltimoSave](#) ()
Carrega o último estado salvo do sistema.
- bool [CarregarSave](#) (std::string arquivo)
Carrega o estado do sistema a partir de um arquivo específico.

6.7.1 Detailed Description

Ponto de entrada central do sistema de monitoramento.

Gerencia o conjunto de linhas de produção, os usuários cadastrados, o controle de sessão (login/logout) e a persistência dos dados.

Definition at line 17 of file [Sistema.hpp](#).

6.7.2 Constructor & Destructor Documentation

6.7.2.1 Sistema()

```
Sistema::Sistema ()
```

Inicializa o sistema sem linhas, usuários ou sessão ativa.

6.7.3 Member Function Documentation

6.7.3.1 AdicionarLinha()

```
void Sistema::AdicionarLinha (  
    int id,  
    std::string desc,  
    std::string local,  
    std::string nome)
```

Cadastra uma nova linha de produção.

Parameters

<i>id</i>	Identificador único da linha.
<i>desc</i>	Descrição da linha.
<i>local</i>	Localização física.
<i>nome</i>	Nome da linha.

Note

Implementa US-001 (Criar Linhas de Produção).

6.7.3.2 AdicionarUsuario()

```
void Sistema::AdicionarUsuario (
    std::string login,
    std::string senha,
    Cargo cargo)
```

Cadastra um novo usuário no sistema.

Parameters

<i>login</i>	Login de acesso.
<i>senha</i>	Senha do usuário.
<i>cargo</i>	Cargo/nível de acesso.

Note

Implementa US-015 (Criar usuários).

6.7.3.3 CarregarSave()

```
bool Sistema::CarregarSave (
    std::string arquivo)
```

Carrega o estado do sistema a partir de um arquivo específico.

Parameters

<i>arquivo</i>	Caminho do arquivo de save.
----------------	-----------------------------

Returns

true se o carregamento foi bem-sucedido.

Note

Implementa US-021 (Carregar dados).

6.7.3.4 CarregarUltimoSave()

```
bool Sistema::CarregarUltimoSave ()
```

Carrega o último estado salvo do sistema.

Returns

true se o carregamento foi bem-sucedido.

Note

Implementa US-021 (Carregar dados).

6.7.3.5 ExibirLinhas()

```
void Sistema::ExibirLinhas ()
```

Exibe no console todas as linhas de produção cadastradas.

Note

Implementa US-003 (Acessar Linhas de Produção).

6.7.3.6 GetLinhas()

```
const std::vector< LinhaDeProducao > & Sistema::GetLinhas () const
```

Returns

Referência constante ao vetor de linhas cadastradas.

6.7.3.7 GetUsuarioLogado()

```
Usuario * Sistema::GetUsuarioLogado () const
```

Returns

Ponteiro para o usuário atualmente logado, ou nullptr se não houver sessão.

6.7.3.8 IdDisponivel()

```
bool Sistema::IdDisponivel (  
    int Id) const
```

Verifica se um ID está disponível para uso.

Parameters

<i>Id</i>	ID a verificar.
-----------	-----------------

Returns

true se o ID não estiver em uso.

6.7.3.9 Login()

```
bool Sistema::Login (
    std::string login,
    std::string senha)
```

Autentica um usuário e inicia a sessão.

Parameters

<i>login</i>	Login do usuário.
<i>senha</i>	Senha do usuário.

Returns

true se as credenciais forem válidas.

Note

Implementa US-017 (Login de usuário).

6.7.3.10 Logout()

```
bool Sistema::Logout ()
```

Encerra a sessão do usuário atual.

Returns

true se havia uma sessão ativa e foi encerrada com sucesso.

Note

Implementa US-022 (Logoff de usuário).

6.7.3.11 ProximoIdDisponivel()

```
int Sistema::ProximoIdDisponivel () const
```

Retorna o próximo ID disponível para cadastro.

Returns

Menor ID inteiro ainda não utilizado.

6.7.3.12 RemoverLinha()

```
void Sistema::RemoverLinha (
    int id)
```

Remove a linha de produção com o ID fornecido.

Parameters

<i>id</i>	Identificador da linha a remover.
-----------	-----------------------------------

Note

Implementa US-002 (Remover Linhas de Produção).

6.7.3.13 RemoverUsuario()

```
void Sistema::RemoverUsuario (
    std::string login)
```

Remove o usuário identificado pelo login.

Parameters

<i>login</i>	Login do usuário a remover.
--------------	-----------------------------

Note

Implementa US-016 (Remover usuários).

6.7.3.14 SalvarAlteracoes()

```
bool Sistema::SalvarAlteracoes ()
```

Persiste o estado atual do sistema em arquivo.

Returns

true se o salvamento foi bem-sucedido.

Note

Implementa US-020 (Persistir dados).

The documentation for this class was generated from the following file:

- include/[Sistema.hpp](#)

6.8 Usuario Class Reference

Representa um usuário do sistema com credenciais e cargo.

```
#include <Usuario.hpp>
```

Public Member Functions

- [Usuario](#) (std::string login, std::string senha, [Cargo](#) cargo)
Constrói um usuário.
- bool [ValidarSenha](#) (std::string senha) const
Verifica se a senha fornecida corresponde à senha do usuário.
- std::string [GetLogin](#) () const
- [Cargo](#) [GetCargo](#) (std::string login)
Retorna o cargo do usuário identificado pelo login.

6.8.1 Detailed Description

Representa um usuário do sistema com credenciais e cargo.

Definition at line 11 of file [Usuario.hpp](#).

6.8.2 Constructor & Destructor Documentation

6.8.2.1 Usuario()

```
Usuario::Usuario (
    std::string login,
    std::string senha,
    Cargo cargo)
```

Constrói um usuário.

Parameters

<i>login</i>	Login de acesso.
<i>senha</i>	Senha do usuário.
<i>cargo</i>	Cargo/nível de acesso.

Note

Implementa US-015 (Criar usuários).

6.8.3 Member Function Documentation

6.8.3.1 GetCargo()

```
Cargo Usuario::GetCargo (
    std::string login)
```

Retorna o cargo do usuário identificado pelo login.

Parameters

<i>login</i>	Login do usuário.
--------------	-------------------

Returns

[Cargo](#) associado ao login.

6.8.3.2 GetLogin()

```
std::string Usuario::GetLogin () const
```

Returns

Login do usuário.

6.8.3.3 ValidarSenha()

```
bool Usuario::ValidarSenha (
    std::string senha) const
```

Verifica se a senha fornecida corresponde à senha do usuário.

Parameters

<i>senha</i>	Senha a validar.
--------------	------------------

Returns

true se a senha estiver correta.

Note

Implementa US-017 (Login de usuário).

The documentation for this class was generated from the following file:

- [include/Usuario.hpp](#)

Chapter 7

File Documentation

7.1 include/Alarme.hpp File Reference

```
#include <string>
#include <vector>
#include "Parametro.hpp"
```

Classes

- class [Alarme](#)
[Alarme](#) associado a um [Parametro](#) de um equipamento.

7.2 Alarme.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef ALARME_HPP
00002 #define ALARME_HPP
00003
00004 #include <string>
00005 #include <vector>
00006 #include "Parametro.hpp"
00007
00014 class Alarme {
00015 private:
00016     std::string _Desc;
00017     bool _Estado;
00018     int _Id;
00019     const Parametro* _Parametro;
00020 public:
00028     Alarme(std::string Desc, int Id, Parametro& p);
00029
00035     bool AtualizarAlarme();
00036
00038     bool GetEstado() const;
00039
00041     void SetEstado(bool Estado);
00042
00044     void SetDesc(std::string desc);
00045 };
00046
00047 #endif
```

7.3 include/Cargo.hpp File Reference

Enumerations

- enum class [Cargo](#) { [ADMIN](#) , [ENGENHEIRO](#) , [TECNICO](#) }
Níveis de acesso dos usuários do sistema.

7.3.1 Enumeration Type Documentation

7.3.1.1 Cargo

```
enum class Cargo [strong]
```

Níveis de acesso dos usuários do sistema.

- ADMIN: Gerencia usuários e a estrutura completa do sistema.
- ENGENHEIRO: Configura linhas, conjuntos e equipamentos.
- TECNICO: Visualiza dados e alarmes.

Enumerator

ADMIN	
ENGENHEIRO	
TECNICO	

Definition at line 11 of file [Cargo.hpp](#).

7.4 Cargo.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef CARGO_HPP
00002 #define CARGO_HPP
00003
00011 enum class Cargo {
00012     ADMIN,
00013     ENGENHEIRO,
00014     TECNICO
00015 };
00016
00017 #endif
```

7.5 include/Conjunto.hpp File Reference

```
#include <string>
#include <vector>
#include "Equipamento.hpp"
```

Classes

- class [Conjunto](#)

Agrupamento lógico de equipamentos dentro de uma [LinhaDeProducao](#).

7.6 Conjunto.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef CONJUNTO_HPP
00002 #define CONJUNTO_HPP
00003
00004 #include <string>
00005 #include <vector>
00006 #include "Equipamento.hpp"
00007
00014 class Conjunto {
00015 private:
00016     int _id;
00017     std::string _desc;
00018     std::string _nome;
00019
00020     std::vector<Equipamento> _Equipamentos;
00021 public:
00027     Conjunto(int id, std::string desc);
00028
00035     void AdicionarEquipamento(int id, std::string desc);
00036
00042     void RemoverEquipamento(int id);
00043
00045     void SetDesc(std::string desc);
00046
00048     void SetNome(std::string nome);
00049
00054     void ExibirEquipamentos() const;
00055
00057     void ExibirAlarmes() const;
00058
00063     void ExibirTudo() const;
00064
00066     int QuantidadeEquipamentos() const;
00067
00069     int QuantidadeAlarmes() const;
00070
00076     std::vector<int> DiagnosticarConjunto() const;
00077
00082     void AtualizarAlarmes();
00083
00085     const std::vector<Equipamento>& GetEquipamentos() const;
00086
00088     int GetId() const;
00089
00091     std::string GetDesc() const;
00092
00094     std::string GetNome() const;
00095 };
00096
00097 #endif

```

7.7 include/Equipamento.hpp File Reference

```

#include <string>
#include <vector>
#include "Parametro.hpp"
#include "Alarme.hpp"

```

Classes

- class [Equipamento](#)

Representa um equipamento físico dentro de um [Conjunto](#).

7.8 Equipamento.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef EQUIPAMENTO_HPP
00002 #define EQUIPAMENTO_HPP
00003
00004 #include <string>
00005 #include <vector>
00006 #include "Parametro.hpp"
00007 #include "Alarme.hpp"
00008
00015 class Equipamento {
00016 private:
00017     int _id;
00018     std::string _desc;
00019
00020     std::vector<Parametro> _Parametros;
00021     std::vector<Alarme> _Alarmes;
00022 public:
00028     Equipamento(int id, std::string desc);
00029
00037     void AdicionarParametro(double ValorAtual, std::string Desc, int id);
00038
00047     void AdicionarParametroComLimites(double ValorAtual, double Max, double Min, std::string Desc);
00048
00053     void RemoverParametro(int id);
00054
00060     std::vector<int> DiagnosticarEquipamento() const;
00061
00066     void AtualizarAlarmes();
00067
00072     void ExibirParametros() const;
00073
00078     void ExibirAlarmes() const;
00079
00081     int QuantidadeParametros() const;
00082
00084     int QuantidadeAlarmes() const;
00085
00087     int GetId() const;
00088
00090     const std::vector<Parametro>& GetParametros() const;
00091
00093     const std::vector<Alarme>& GetAlarmes() const;
00094
00096     void SetDesc(std::string desc);
00097 };
00098
00099 #endif

```

7.9 include/LinhaDeProducao.hpp File Reference

```

#include <vector>
#include <string>
#include "Conjunto.hpp"

```

Classes

- class [LinhaDeProducao](#)
Representa uma linha de produção industrial.

7.10 LinhaDeProducao.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef LINHADEPRODUCAO_HPP

```

```

00002 #define LINHADEPRODUCAO_HPP
00003
00004 #include <vector>
00005 #include <string>
00006
00007 #include "Conjunto.hpp"
00008
00015 class LinhaDeProducao {
00016 private:
00017     int _id;
00018     std::string _nome;
00019     std::string _local;
00020     std::string _desc;
00021
00022     std::vector<Conjunto> _Conjuntos;
00023 public:
00031     LinhaDeProducao(int id, std::string desc, std::string local, std::string nome);
00032
00039     void AdicionarConjunto(int id, std::string desc);
00040
00046     void RemoverConjunto(int id);
00047
00052     void ExibirConjuntos() const;
00053
00055     void ExibirAlarmes() const;
00056
00061     void ExibirTudo() const;
00062
00064     int QuantidadeConjuntos() const;
00065
00067     int QuantidadeAlarmes() const;
00068
00074     std::vector<int> DiagnosticarLinha() const;
00075
00080     void AtualizarAlarmes();
00081
00083     void SetDesc(std::string desc);
00084
00086     void SetLocal(std::string local);
00087
00089     void SetNome(std::string nome);
00090
00092     const std::vector<Conjunto>& GetConjuntos() const;
00093
00095     int GetId() const;
00096
00098     std::string GetDesc() const;
00099
00101     std::string GetLocal() const;
00102
00104     std::string GetNome() const;
00105 };
00106
00107 #endif

```

7.11 include/Log.hpp File Reference

```
#include <string>
```

Classes

- class [Log](#)

Registra eventos e operações do sistema em arquivo de texto.

7.12 Log.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef LOG_HPP
```

```

00002 #define LOG_HPP
00003
00004 #include <string>
00005
00009 class Log {
00010 private:
00011     std::string _arquivo;
00012 public:
00017     Log(const std::string& arquivo);
00018
00025     bool Registrar(const std::string& mensagem) const;
00026
00028     std::string GetArquivoLog() const;
00029
00035     bool SetArquivoLog(const std::string& arquivo);
00036 };
00037
00038 #endif

```

7.13 include/Parametro.hpp File Reference

```

#include <string>
#include <vector>

```

Classes

- class [Parametro](#)

Representa um parâmetro monitorado de um equipamento.

7.14 Parametro.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef PARAMETRO_HPP
00002 #define PARAMETRO_HPP
00003
00004 #include <string>
00005 #include <vector>
00006
00013 class Parametro {
00014 private:
00015     int _id;
00016     std::string _Desc;
00017
00018     double _ValorAtual;
00019     std::vector<double> _Hist;
00020
00021     double _Max;
00022     double _Min;
00023
00024     bool _Estado;
00025 public:
00032     Parametro(double ValorAtual, std::string Desc, int id);
00033
00043     Parametro(double ValorAtual, double Max, double Min, std::string Desc, int id);
00044
00050     bool ValorValido(double Valor) const;
00051
00057     void SetValorAtual(double Valor);
00058
00064     void SetMax(double Valor);
00065
00071     void SetMin(double Valor);
00072
00074     void SetDesc(std::string Desc);
00075
00081     bool DiagnosticarParametro();
00082
00084     double GetValorAtual() const;

```



```

00085
00087     double GetMax() const;
00088
00090     double GetMin() const;
00091
00093     std::string GetDesc() const;
00094
00096     bool GetEstado() const;
00097
00102     const std::vector<double>& GetHist() const;
00103
00105     int GetId();
00106
00108     void ResetarLimites();
00109
00111     void ResetarHistorico();
00112 };
00113
00114 #endif

```

7.15 include/Sistema.hpp File Reference

```

#include <vector>
#include <string>
#include "LinhaDeProducao.hpp"
#include "Usuario.hpp"
#include "Cargo.hpp"

```

Classes

- class [Sistema](#)
Ponto de entrada central do sistema de monitoramento.

7.16 Sistema.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef SISTEMA_HPP
00002 #define SISTEMA_HPP
00003
00004 #include <vector>
00005 #include <string>
00006
00007 #include "LinhaDeProducao.hpp"
00008 #include "Usuario.hpp"
00009 #include "Cargo.hpp"
00010
00017 class Sistema {
00018 private:
00019     std::vector<LinhaDeProducao> _Linhas;
00020     std::vector<Usuario> _Usuarios;
00021     std::vector<int> _IdsUtilizados;
00022
00023     Usuario* _UsuarioLogado;
00024
00026     void AdicionarId(int Id);
00027
00029     void RemoverId(int Id);
00030 public:
00032     Sistema();
00033
00042     void AdicionarLinha(int id, std::string desc, std::string local, std::string nome);
00043
00049     void RemoverLinha(int id);
00050
00052     const std::vector<LinhaDeProducao>& GetLinhas() const;
00053

```

```

00058     void ExibirLinhas();
00059
00067     void AdicionarUsuario(std::string login, std::string senha, Cargo cargo);
00068
00074     void RemoverUsuario(std::string login);
00075
00080     int ProximoIdDisponivel() const;
00081
00087     bool IdDisponivel(int Id) const;
00088
00096     bool Login(std::string login, std::string senha);
00097
00103     bool Logout();
00104
00106     Usuario* GetUsuarioLogado() const;
00107
00113     bool SalvarAlteracoes();
00114
00120     bool CarregarUltimoSave();
00121
00128     bool CarregarSave(std::string arquivo);
00129 };
00130
00131 #endif

```

7.17 include/Usuario.hpp File Reference

```

#include <string>
#include "Cargo.hpp"

```

Classes

- class [Usuario](#)
Representa um usuário do sistema com credenciais e cargo.

7.18 Usuario.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef USUARIO_HPP
00002 #define USUARIO_HPP
00003
00004 #include <string>
00005
00006 #include "Cargo.hpp"
00007
00011 class Usuario {
00012 private:
00013     std::string _login;
00014     std::string _senha;
00015     Cargo _cargo;
00016 public:
00024     Usuario(std::string login, std::string senha, Cargo cargo);
00025
00032     bool ValidarSenha(std::string senha) const;
00033
00035     std::string GetLogin() const;
00036
00042     Cargo GetCargo(std::string login);
00043 };
00044
00045 #endif

```

7.19 README.md File Reference

Index

- AdicionarConjunto
 - LinhaDeProducao, [24](#)
- AdicionarEquipamento
 - Conjunto, [14](#)
- AdicionarLinha
 - Sistema, [36](#)
- AdicionarParametro
 - Equipamento, [19](#)
- AdicionarParametroComLimites
 - Equipamento, [19](#)
- AdicionarUsuario
 - Sistema, [36](#)
- ADMIN
 - Cargo.hpp, [44](#)
- Alarme, [11](#)
 - Alarme, [12](#)
 - AtualizarAlarme, [12](#)
 - GetEstado, [12](#)
 - SetDesc, [12](#)
 - SetEstado, [13](#)
- AtualizarAlarme
 - Alarme, [12](#)
- AtualizarAlarmes
 - Conjunto, [15](#)
 - Equipamento, [20](#)
 - LinhaDeProducao, [24](#)
- Cargo
 - Cargo.hpp, [44](#)
- Cargo.hpp
 - ADMIN, [44](#)
 - Cargo, [44](#)
 - ENGENHEIRO, [44](#)
 - TECNICO, [44](#)
- CarregarSave
 - Sistema, [37](#)
- CarregarUltimoSave
 - Sistema, [37](#)
- Conjunto, [13](#)
 - AdicionarEquipamento, [14](#)
 - AtualizarAlarmes, [15](#)
 - Conjunto, [14](#)
 - DiagnosticarConjunto, [15](#)
 - ExibirAlarmes, [15](#)
 - ExibirEquipamentos, [15](#)
 - ExibirTudo, [15](#)
 - GetDesc, [16](#)
 - GetEquipamentos, [16](#)
 - GetId, [16](#)
 - GetNome, [16](#)
 - QuantidadeAlarmes, [16](#)
 - QuantidadeEquipamentos, [17](#)
 - RemoverEquipamento, [17](#)
 - SetDesc, [17](#)
 - SetNome, [17](#)
- DiagnosticarConjunto
 - Conjunto, [15](#)
- DiagnosticarEquipamento
 - Equipamento, [20](#)
- DiagnosticarLinha
 - LinhaDeProducao, [24](#)
- DiagnosticarParametro
 - Parametro, [31](#)
- ENGENHEIRO
 - Cargo.hpp, [44](#)
- Equipamento, [18](#)
 - AdicionarParametro, [19](#)
 - AdicionarParametroComLimites, [19](#)
 - AtualizarAlarmes, [20](#)
 - DiagnosticarEquipamento, [20](#)
 - Equipamento, [19](#)
 - ExibirAlarmes, [20](#)
 - ExibirParametros, [21](#)
 - GetAlarmes, [21](#)
 - GetId, [21](#)
 - GetParametros, [21](#)
 - QuantidadeAlarmes, [21](#)
 - QuantidadeParametros, [22](#)
 - RemoverParametro, [22](#)
 - SetDesc, [22](#)
- ExibirAlarmes
 - Conjunto, [15](#)
 - Equipamento, [20](#)
 - LinhaDeProducao, [24](#)
- ExibirConjuntos
 - LinhaDeProducao, [25](#)
- ExibirEquipamentos
 - Conjunto, [15](#)
- ExibirLinhas
 - Sistema, [37](#)
- ExibirParametros
 - Equipamento, [21](#)
- ExibirTudo
 - Conjunto, [15](#)
 - LinhaDeProducao, [25](#)
- GetAlarmes
 - Equipamento, [21](#)

- GetArquivoLog
 - Log, [29](#)
- GetCargo
 - Usuario, [42](#)
- GetConjuntos
 - LinhaDeProducao, [25](#)
- GetDesc
 - Conjunto, [16](#)
 - LinhaDeProducao, [25](#)
 - Parametro, [31](#)
- GetEquipamentos
 - Conjunto, [16](#)
- GetEstado
 - Alarme, [12](#)
 - Parametro, [32](#)
- GetHist
 - Parametro, [32](#)
- GetId
 - Conjunto, [16](#)
 - Equipamento, [21](#)
 - LinhaDeProducao, [25](#)
 - Parametro, [32](#)
- GetLinhas
 - Sistema, [38](#)
- GetLocal
 - LinhaDeProducao, [26](#)
- GetLogin
 - Usuario, [42](#)
- GetMax
 - Parametro, [32](#)
- GetMin
 - Parametro, [32](#)
- GetNome
 - Conjunto, [16](#)
 - LinhaDeProducao, [26](#)
- GetParametros
 - Equipamento, [21](#)
- GetUsuarioLogado
 - Sistema, [38](#)
- GetValorAtual
 - Parametro, [33](#)
- IdDisponivel
 - Sistema, [38](#)
- include Directory Reference, [9](#)
- include/Alarme.hpp, [43](#)
- include/Cargo.hpp, [44](#)
- include/Conjunto.hpp, [44](#), [45](#)
- include/Equipamento.hpp, [45](#), [46](#)
- include/LinhaDeProducao.hpp, [46](#)
- include/Log.hpp, [47](#)
- include/Parametro.hpp, [48](#)
- include/Sistema.hpp, [49](#)
- include/Usuario.hpp, [50](#)
- LinhaDeProducao, [22](#)
 - AdicionarConjunto, [24](#)
 - AtualizarAlarmes, [24](#)
 - DiagnosticarLinha, [24](#)
 - ExibirAlarmes, [24](#)
 - ExibirConjuntos, [25](#)
 - ExibirTudo, [25](#)
 - GetConjuntos, [25](#)
 - GetDesc, [25](#)
 - GetId, [25](#)
 - GetLocal, [26](#)
 - GetNome, [26](#)
 - LinhaDeProducao, [23](#)
 - QuantidadeAlarmes, [26](#)
 - QuantidadeConjuntos, [26](#)
 - RemoverConjunto, [26](#)
 - SetDesc, [27](#)
 - SetLocal, [27](#)
 - SetNome, [27](#)
- Log, [28](#)
 - GetArquivoLog, [29](#)
 - Log, [28](#)
 - Registrar, [29](#)
 - SetArquivoLog, [29](#)
- Login
 - Sistema, [38](#)
- Logout
 - Sistema, [39](#)
- Parametro, [30](#)
 - DiagnosticarParametro, [31](#)
 - GetDesc, [31](#)
 - GetEstado, [32](#)
 - GetHist, [32](#)
 - GetId, [32](#)
 - GetMax, [32](#)
 - GetMin, [32](#)
 - GetValorAtual, [33](#)
 - Parametro, [31](#)
 - ResetarHistorico, [33](#)
 - ResetarLimites, [33](#)
 - SetDesc, [33](#)
 - SetMax, [33](#)
 - SetMin, [34](#)
 - SetValorAtual, [34](#)
 - ValorValido, [34](#)
- ProximoIdDisponivel
 - Sistema, [39](#)
- QuantidadeAlarmes
 - Conjunto, [16](#)
 - Equipamento, [21](#)
 - LinhaDeProducao, [26](#)
- QuantidadeConjuntos
 - LinhaDeProducao, [26](#)
- QuantidadeEquipamentos
 - Conjunto, [17](#)
- QuantidadeParametros
 - Equipamento, [22](#)
- README.md, [50](#)
- Registrar
 - Log, [29](#)

- RemoverConjunto
 - LinhaDeProducao, [26](#)
- RemoverEquipamento
 - Conjunto, [17](#)
- RemoverLinha
 - Sistema, [39](#)
- RemoverParametro
 - Equipamento, [22](#)
- RemoverUsuario
 - Sistema, [40](#)
- ResetarHistorico
 - Parametro, [33](#)
- ResetarLimites
 - Parametro, [33](#)
- SalvarAlteracoes
 - Sistema, [40](#)
- SetArquivoLog
 - Log, [29](#)
- SetDesc
 - Alarme, [12](#)
 - Conjunto, [17](#)
 - Equipamento, [22](#)
 - LinhaDeProducao, [27](#)
 - Parametro, [33](#)
- SetEstado
 - Alarme, [13](#)
- SetLocal
 - LinhaDeProducao, [27](#)
- SetMax
 - Parametro, [33](#)
- SetMin
 - Parametro, [34](#)
- SetNome
 - Conjunto, [17](#)
 - LinhaDeProducao, [27](#)
- SetValorAtual
 - Parametro, [34](#)
- Sistema, [35](#)
 - AdicionarLinha, [36](#)
 - AdicionarUsuario, [36](#)
 - CarregarSave, [37](#)
 - CarregarUltimoSave, [37](#)
 - ExibirLinhas, [37](#)
 - GetLinhas, [38](#)
 - GetUsuarioLogado, [38](#)
 - IdDisponivel, [38](#)
 - Login, [38](#)
 - Logout, [39](#)
 - ProximoldDisponivel, [39](#)
 - RemoverLinha, [39](#)
 - RemoverUsuario, [40](#)
 - SalvarAlteracoes, [40](#)
 - Sistema, [36](#)
- Sistema de Tolerância de Falha em Fábricas, [1](#)
- TECNICO
 - Cargo.hpp, [44](#)
- Usuario, [41](#)
 - GetCargo, [42](#)
 - GetLogin, [42](#)
 - Usuario, [41](#)
 - ValidarSenha, [42](#)
- ValidarSenha
 - Usuario, [42](#)
- ValorValido
 - Parametro, [34](#)